

SEQUENTIAL SYSTEMS

Phase III: Asynchronous Cascades

(Ripple Counters & Truncation)

A Comprehensive Master Manual

Topics Covered:

The Asynchronous Architecture
The 3-Bit Binary Up-Counter
Propagation Delay & Timing Analysis
Maximum Frequency Mathematics (f_{max})
Directional Control (Down Counters)
Modulus Truncation (Mod-N Design)
The Decade Counter (BCD)
Rigorous TikZ Vector Schematics

Format: Pure $\text{\LaTeX}$ Typesetting (Expanded Edition)

EE/PC/B/T/225 | Jadavpur University

Contents

1	The Asynchronous Architecture	2
1.1	Introduction to Counting	2
1.2	The Concept of the Ripple	2
1.3	The Core Engine: The T-Flip-Flop	4
1.3.1	The Triggering Edge	4
2	The 3-Bit Binary Up-Counter	5
2.1	Hardware Synthesis	5
2.1.1	Architectural Schematic	5
2.2	State Transition Analysis	6
3	Timing and The Propagation Crisis	8
3.1	The Ripple Timing Diagram	8
3.2	The Propagation Delay Limitation	9
3.2.1	Decoding Glitches & Frequency Ceilings	9
4	Modulus Truncation (Mod-N Design)	11
4.1	The Concept of the Truncated Sequence	11
4.2	Designing a Mod-6 Ripple Counter	11
4.2.1	The CLEAR Override Strategy	11
4.2.2	Mod-6 Architectural Schematic	13

5 The Decade Counter (BCD)	14
5.1 The Mod-10 Architecture	14
5.1.1 The Truncation Logic	14
5.1.2 Commercial Implementation: IC 7490	15

Chapter 1

The Asynchronous Architecture

1.1 Introduction to Counting

A counter is one of the most fundamental and ubiquitous sequential networks in digital electronics. At its core, a counter is an array of memory elements (Flip-Flops) that advance through a predetermined sequence of states upon the application of an external stimulus (a clock pulse).

Because these clock pulses occur at precisely known intervals, counters serve as the fundamental basis for measuring time, measuring frequency, and orchestrating the chronological sequence of events within a microprocessor.

1.2 The Concept of the Ripple

Counters are broadly classified into two categories based on how the master clock signal is distributed to the individual flip-flops: **Synchronous** and **Asynchronous**.

In an **Asynchronous Counter**, the master clock from the oscillator is strictly applied to *only* the first flip-flop in the array (the Least Significant Bit, or LSB).

The output of the LSB flip-flop then physically serves as the clock input for the second flip-flop. The output of the second serves as the clock for the third, and so

on. Because the clock signal must physically cascade from one flip-flop to the next, the state changes do not happen simultaneously. Instead, they "ripple" sequentially through the circuit. Thus, asynchronous counters are universally referred to as **Ripple Counters**.

1.3 The Core Engine: The T-Flip-Flop

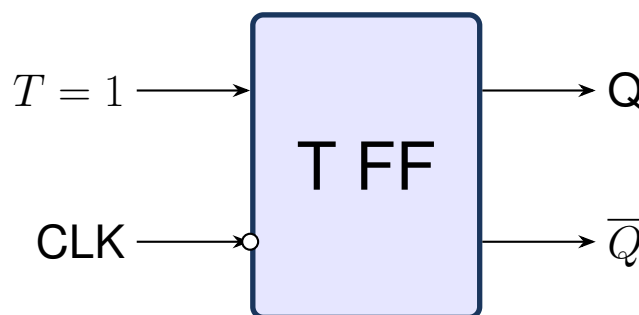
To construct a binary ripple counter, we rely heavily on the **Toggle (T) Flip-Flop**. If a T flip-flop is not available, we synthesize it by tying the J and K pins of a JK flip-flop permanently to Logic 1 ($+V_{cc}$).

By permanently asserting $T = 1$, the flip-flop enters a permanent toggle mode. It will invert its current state ($Q_{n+1} = \overline{Q}_n$) exactly once every time an active edge arrives at its clock pin.

1.3.1 The Triggering Edge

The choice of whether the flip-flop triggers on the Positive (Rising) Edge or the Negative (Falling) Edge fundamentally alters the behavior of the counter.

- **Negative-Edge Triggered:** The flip-flop toggles when the clock drops from 1 to 0. This creates an **Up-Counter**.
- **Positive-Edge Triggered:** The flip-flop toggles when the clock rises from 0 to 1. This creates a **Down-Counter**.



The Fundamental Ripple Engine

Chapter 2

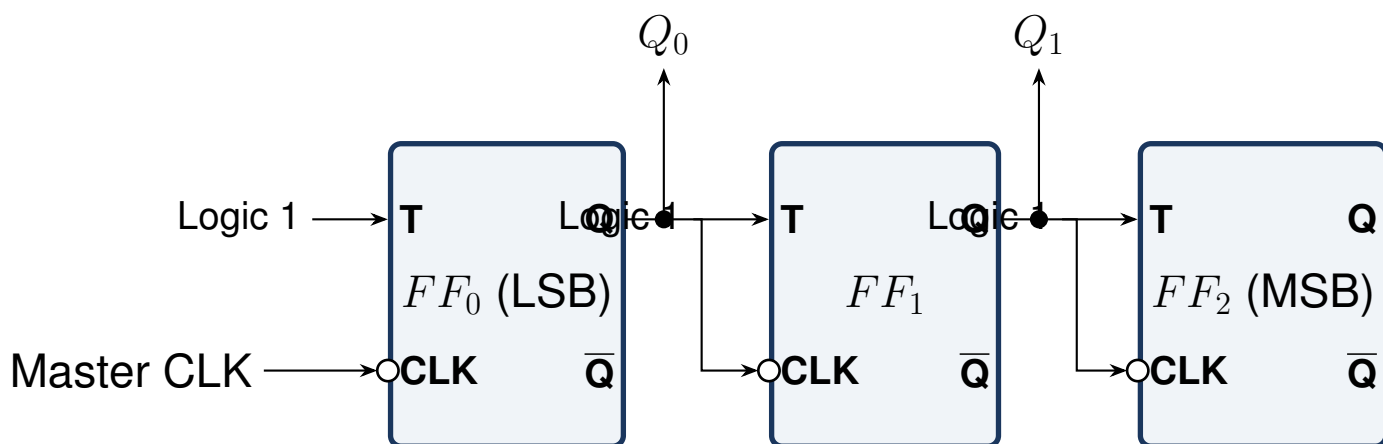
The 3-Bit Binary Up-Counter

2.1 Hardware Synthesis

Let us design a 3-bit Asynchronous Up-Counter. Since the counter is 3 bits wide, it will possess $2^3 = 8$ distinct states, counting from 000 (0) to 111 (7) before overflowing back to 000. Such a counter is formally classified as a **Modulo-8 (Mod-8) Ripple Counter**.

We require three Negative-Edge Triggered T-Flip-Flops, designated as FF_0 (LSB), FF_1 , and FF_2 (MSB).

2.1.1 Architectural Schematic



2.2 State Transition Analysis

To mathematically verify that this asynchronous topology functions correctly as an up-counter, we construct the State Transition Table.

Because we are utilizing negative-edge triggered flip-flops:

- FF_0 toggles on every falling edge of the **Master CLK**.
- FF_1 toggles only when Q_0 transitions from **1 to 0** (falling edge).
- FF_2 toggles only when Q_1 transitions from **1 to 0** (falling edge).

Clock Pulse	Q_2 (MSB)	Q_1	Q_0 (LSB)	Decimal
0 (Initial)	0	0	0	0
1	0	0	1	1
2	0	1	0	2
3	0	1	1	3
4	1	0	0	4
5	1	0	1	5
6	1	1	0	6
7	1	1	1	7
8 (Overflow)	0	0	0	0

Execution Logic Example (Pulse 3 → 4): At pulse 3, the state is 011. When pulse 4 arrives (falling edge), Q_0 toggles from 1 to 0. Because Q_0 fell from 1 to 0, it

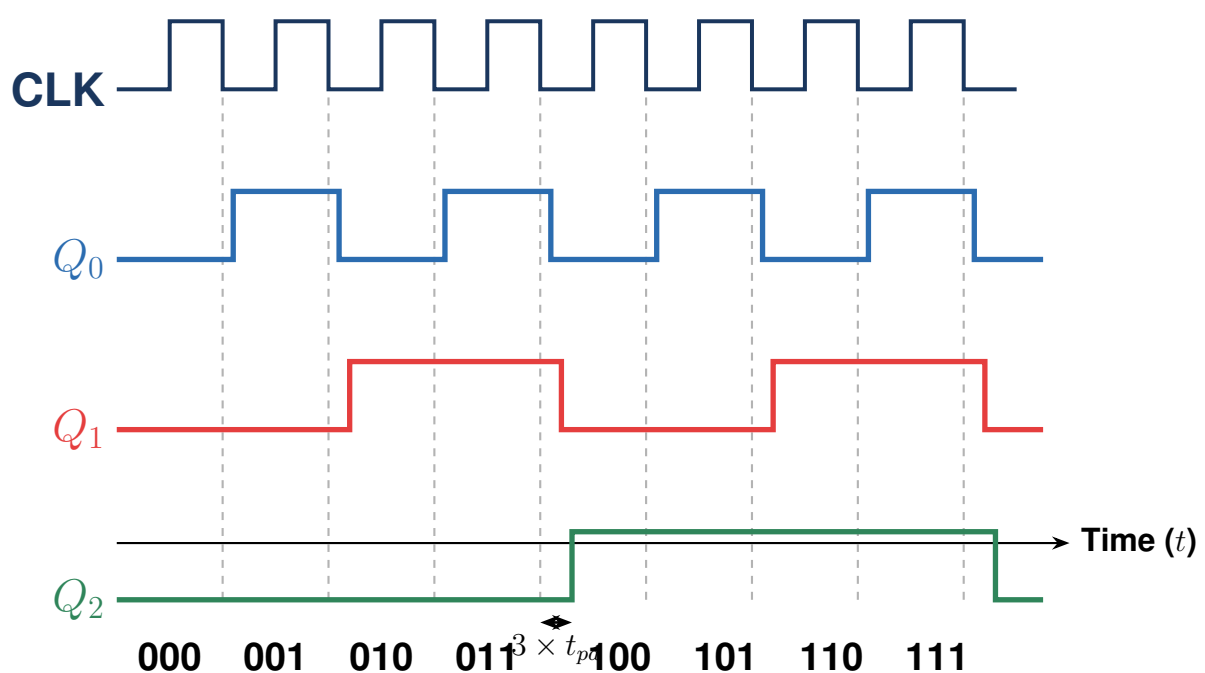
acts as a falling edge clock for FF_1 . Thus, Q_1 toggles from 1 to 0. Because Q_1 fell from 1 to 0, it acts as a falling edge clock for FF_2 . Thus, Q_2 toggles from 0 to 1. The final state successfully ripples to 100 (Decimal 4).

Chapter 3

Timing and The Propagation Crisis

3.1 The Ripple Timing Diagram

The true nature of an asynchronous counter can only be understood by examining its temporal behavior. The diagram below illustrates the exact microsecond-by-microsecond logic transitions. Notice how Q_0 dictates the rhythm of Q_1 , and Q_1 dictates the rhythm of Q_2 .



3.2 The Propagation Delay Limitation

Notice the visual "staircase" effect in the timing diagram above. This is the hallmark of the Ripple Counter.

In physical silicon hardware, logic gates are not infinitely fast. It takes a tiny fraction of a second (typically 10 to 30 nanoseconds, denoted as t_{pd}) for the transistors inside the flip-flop to react to the clock pulse and update the Q output.

Because an asynchronous counter relies on the previous flip-flop's output to act as the clock for the next stage, these delays **accumulate linearly**.

For an N-bit ripple counter, the time it takes for the Most Significant Bit (MSB) to finally settle into its correct state after a master clock pulse is:

$$T_{settle} = N \times t_{pd} \quad (3.1)$$

3.2.1 Decoding Glitches & Frequency Ceilings

During this settling time, the counter outputs temporary, invalid states (known as "garbage" states or **Decoding Glitches**). If external hardware reads the counter during this ripple window, it will receive fatally flawed data.

To prevent the next clock pulse from arriving before the previous ripple has fully completed its journey to the MSB, the master clock frequency (f_{max}) must be severely restricted:

$$f_{max} = \frac{1}{N \times t_{pd}} \quad (3.2)$$

Example: If a 4-bit ripple counter uses flip-flops with a 25ns propagation delay, the

maximum safe operating frequency is exactly:

$$f_{max} = \frac{1}{4 \times 25\text{ns}} = \frac{1}{100 \times 10^{-9}} = 10 \text{ MHz}$$

This is why synchronous (parallel) counters are mandated for modern, high-frequency GHz processors.

Chapter 4

Modulus Truncation (Mod-N Design)

4.1 The Concept of the Truncated Sequence

A natural N-bit binary counter counts continuously up to $2^N - 1$. A 3-bit counter is naturally a Mod-8 counter (states 0 through 7).

However, many engineering applications require non-binary moduli. For instance, a digital clock requires a Mod-60 counter (for minutes/seconds) and a Mod-12 counter (for hours). To achieve this, we must deliberately abort the natural counting sequence and forcefully reset the system to 0 when it reaches the target state.

4.2 Designing a Mod-6 Ripple Counter

We desire a counter that counts exactly: 0, 1, 2, 3, 4, 5... and then resets back to 0. It must possess exactly 6 distinct valid states.

4.2.1 The CLEAR Override Strategy

Because $2^2 < 6 \leq 2^3$, we fundamentally require 3 Flip-Flops. The binary sequence proceeds naturally up to state 5 (101).

The absolute instant the counter attempts to transition into state 6 (Binary 110,

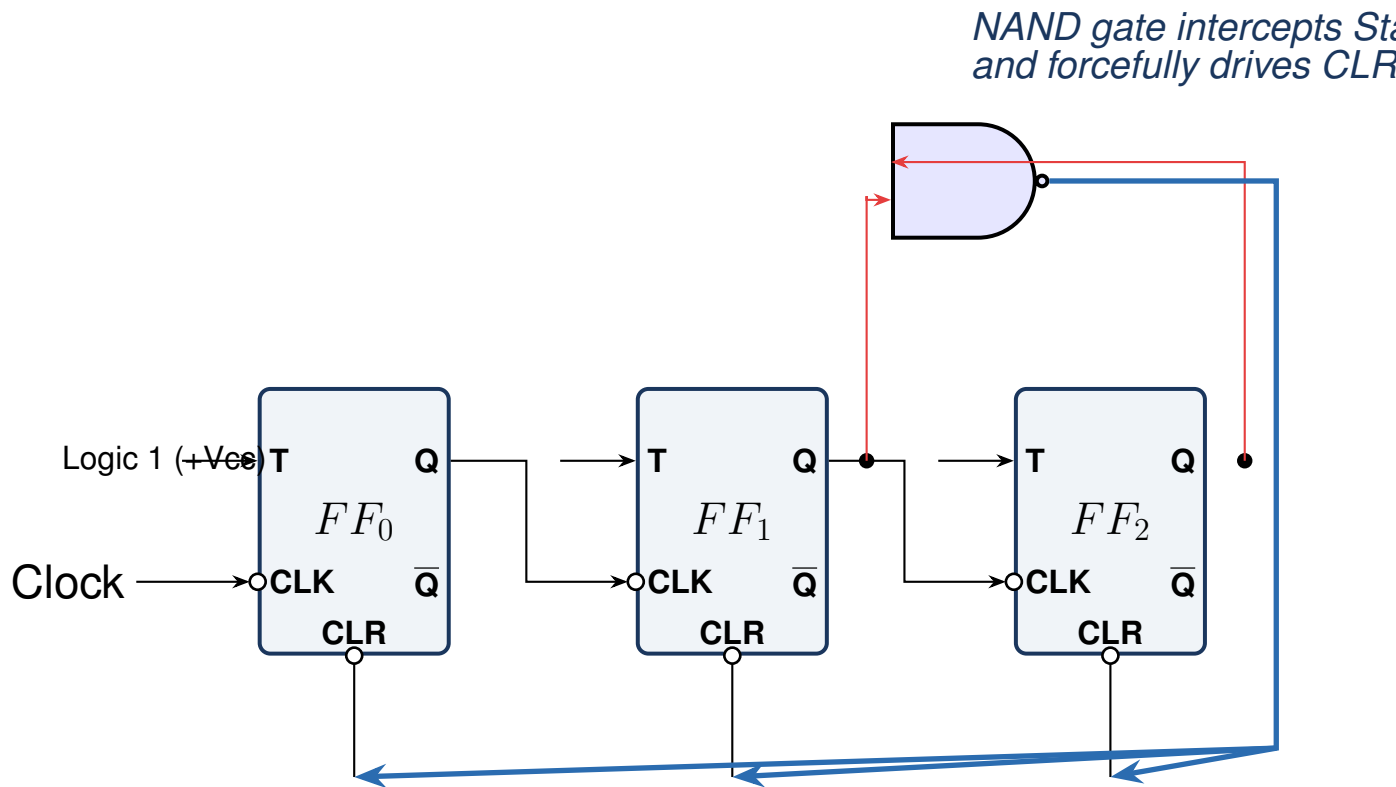
where $Q_2 = 1$ and $Q_1 = 1$), we must intercept this state and trigger a catastrophic reset.

Most flip-flops are equipped with an asynchronous, active-low **CLEAR** ($\overline{\text{CLR}}$) pin. Pulling this pin LOW (0V) instantly forces $Q = 0$, bypassing the clock entirely.

To trigger this automatically upon reaching State 6 ($Q_2 = 1, Q_1 = 1$), we deploy a 2-input **NAND gate**.

- When $Q_2 = 1$ and $Q_1 = 1$, the NAND gate evaluates $1 \cdot 1$ and outputs a **0**.
- This **0** is routed directly into the $\overline{\text{CLR}}$ pins of all flip-flops.
- The flip-flops are instantly reset to 000, wiping out the 6 before external hardware can register it.

4.2.2 Mod-6 Architectural Schematic



Chapter 5

The Decade Counter (BCD)

5.1 The Mod-10 Architecture

One of the most commercially prevalent variations of the truncated ripple counter is the **Decade Counter**, or BCD (Binary Coded Decimal) counter. It counts strictly from 0 to 9, mapping perfectly to human numerical display systems (seven-segment displays).

Because $2^3 < 10 \leq 2^4$, the counter inherently demands **4 Flip-Flops** (Q_3, Q_2, Q_1, Q_0).

5.1.1 The Truncation Logic

The natural sequence of 4 bits counts to 15 (1111). We must abort the sequence at exactly State 10. In binary, State 10 is expressed as **1010** ($Q_3 = 1, Q_2 = 0, Q_1 = 1, Q_0 = 0$).

To trigger the asynchronous reset sequence, we employ the same NAND gate strategy. We isolate the bits that distinguish the number 10. We wire the inputs of the NAND gate to the Q_3 (value 8) and Q_1 (value 2) flip-flop outputs.

When the counter hits 10, both Q_3 and Q_1 are HIGH. The NAND gate outputs a 0, instantly flushing the entire flip-flop array back to 0000.

5.1.2 Commercial Implementation: IC 7490

This exact architecture is physically silicon-etched into the famous 7490 Decade Counter IC. It features a divide-by-2 section (a single flip-flop) and a separate divide-by-5 section (three flip-flops with internal Mod-5 NAND truncation). To achieve the full BCD decade count, the engineer physically wires the output of the first stage into the clock of the second stage, cascading them into a $2 \times 5 = 10$ modulus chain.

***** END OF PHASE III MASTER
MANUAL *****